

Evaluating OPC UA Security: Insights from DoS Attack Scenarios

Raiff S. Silva
Virtus-CC UFCG
Campina Grande, Brazil
raiff.silva@virtus-cc.ufcg.edu.br

Igor A. de Souza
Virtus-CC UFCG
Campina Grande, Brazil
igor.souza@virtus-cc.ufcg.edu.br

Moisés T. da Silva
UABJ UFRPE
Belo Jardim, Brazil
moises.tavares@ufrpe.br

Thiago A. M. Euzebio
Virtus-CC UFCG
Campina Grande, Brazil
thiago.euzebio@virtus-cc.ufcg.edu.br

Marcelo L. de Lima
CENPES PETROBRAS
Rio de Janeiro, Brazil
marceloll@petrobras.com.br

George Acioli Júnior
LIEC UFCG
Campina Grande, Brazil
georgeacioli@dee.ufcg.edu.br

Péricles R. Barros
LIEC UFCG
Campina Grande, Brazil
prbarros@dee.ufcg.edu.br

Abstract—This initial study examines the impact of Denial of Service (DoS) attacks on OPC UA communications, focusing on their effects under varying security configurations. Using both the hping3 tool and Claroty's OPC UA Exploitation Framework, the study found that SecurityMode None permits substantial CPU and RAM resource consumption only with the Exploitation Framework attack, while hping3 had no such impact. Security configurations employing Sign or Sign and Encrypt effectively blocked the attack by rejecting connection attempts. These results highlight the critical role of robust protective mechanisms in defending systems with OPC UA communication against DoS threats.

Index Terms—OPC UA, DoS, Security Configurations.

I. INTRODUCTION

Contemporary automation systems rely on industrial communication networks to support control, monitoring, quality evaluation, maintenance, and the integration of management systems within production and manufacturing operations. In recent years, the Open Platform Communications Unified Architecture (OPC UA) has gained prominence as a leading protocol, demonstrated by its increasing adoption across various applications [1]. Its popularity stems from its platform independence, robust security features, and ability to continuously exchange data across diverse systems and devices.

The most critical aspect of information security in industrial control systems is availability, and one of the most severe threats to this is a DoS attack [2]. A DoS attack involves overwhelming a system, network, or service with excessive traffic or requests, effectively disrupting access for legitimate users. In industrial settings, a DoS attack can, for instance, disrupt communication between the control room and the shop floor, leaving operators unaware of real-time conditions in the production plant.

As OPC UA becomes more prevalent in industrial environments, the need for security studies and analyses has grown. The study in [3] investigates the security challenges of using OPC UA in Industry 4.0, focusing on the threats posed by eavesdropping and message flooding attacks in a real testbed. The thesis in [4] examines OPC UA security in Industrial Internet of Things (IIoT), assessing the impact

of cyberattacks such as packet sniffing, man-in-the-middle, and DoS, and suggesting countermeasures. Meanwhile, [5] validates DoS attack scenarios on OPC UA using flooding, highlighting its severe effects and proposing a framework to identify vulnerabilities. Although these studies emphasize DoS attacks, they primarily rely on basic tools like hping3, overlooking the use of more advanced and OPC-specific tools for analysis.

This paper presents an initial case study of a sophisticated DoS attack targeting OPC UA client-server communication, utilizing the advanced method developed by Claroty Team82 [6]. This approach, specifically developed for OPC UA communications, surpasses traditional tools like hping3 in effectiveness. The study provides a preliminary analysis of the attack's impact on system performance by monitoring CPU utilization and memory usage across three OPC UA security configurations: no security, user authentication, and sign-and-encrypt. The findings highlight the practical implications of these configurations in mitigating DoS attacks and emphasize the critical need for robust, tailored cybersecurity measures in OPC UA environments.

II. BACKGROUND

In this section, the fundamentals of the OPC UA standard and the main security aspects are presented. In addition, general aspects of the DoS attack are discussed.

A. OPC UA overview

The OPC UA standard provides a platform-independent framework for exchanging data from the factory floor to the enterprise level. Due to its scalable architecture, OPC UA is suitable for future use in smart embedded devices of Industry 4.0 [7]. A promising application of OPC UA is in the Open Process Automation Standard (OPAS) through the OPAS Connectivity Framework (OCF). The OCF defines the mechanisms for handling the information flows between instances of OPAS Distributed Control Nodes (DCNs) utilizing a standard OPC UA communication model [8].

OPC UA defines two communication architectures: client-server and PubSub (Publisher-Subscriber). Regardless of the

architecture used, information is structured using an object-oriented model, where objects are called "nodes". The OPC UA model defines node objects in terms of variables, methods and events. To access the information stored on the nodes, a set of services are provided, such as read/write attributes and subscription services to monitor specific items [9].

The primary transport protocol is the Unified Architecture Transmission Control Protocol (UA TCP), designed for efficient network communication. UA TCP messages include a "Hello" message from the client to the server to establish a connection, followed by an acknowledgement from the server confirming the buffer and message sizes [9].

In IIoT-based applications, OPC UA enables secure interoperability between several data sources and networks. Thus, OPC UA provides security mechanisms against well-known vulnerabilities that could potentially be exploited by malicious actors. The different components of an OPC UA server for defining connection rules and configurations are shown in Figure 1. In a client-server connection, the client must prove possession of a `UserIdentityToken` [10]. The server endpoint determines which `UserIdentityToken` is allowed, this is done through the `UserTokenPolicy` associated with the `EndpointDescription`. The `UserTokenPolicy` also specifies which `SecurityPolicy` will be used when performing the encryption and signing processes.

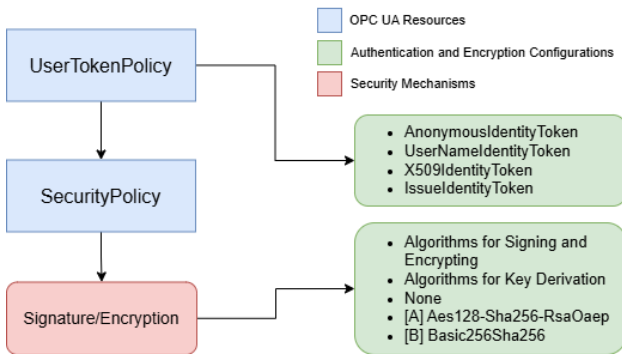


Fig. 1. OPC UA server security configuration components.

B. OPC UA Security and Performance Trade-off

In the process of setting up a client-server connection, the use of appropriate security rules depends on the environment in which the device operates. According to Dahlmanns et al. [11] there is a critical gap between the knowledge and the effort required to configure security in IIoT implementations with secure configuration needs based on individual demands.

The configuration task is not limited to enabling all available security mechanisms or none at all. The impact on performance of OPC UA client-server communication, as illustrated in Figure 2, demonstrates the trade-offs when adding functionalities such as signing and encryption to ensure communication with confidentiality and integrity guarantees.

In addition to performance, the security analysis conducted by the Federal Office for Information Security, Germany's

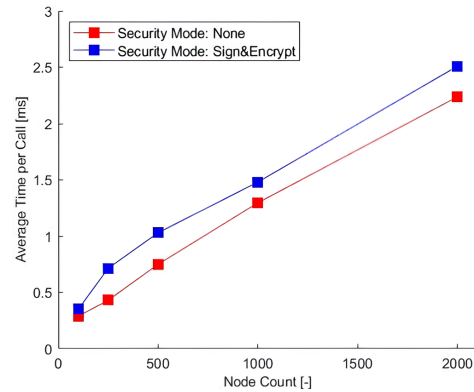


Fig. 2. Example of the average time per read service call in a client-server OPC UA communication in localhost using the performance of UAExpert tool.

national cybersecurity agency [12], demonstrates effective protection against threats to confidentiality and integrity when signing and encryption mechanisms are employed. However, for availability, the protection presents limitations, requiring additional security measures beyond the scope of OPC UA. In some scenarios, individual configuration of OPC UA devices is required, as well as the need to provide availability without concerns about confidentiality and integrity. In these cases, it is important to evaluate the protective capacity of the various security configurations, and the cost of performance degradation.

C. OPC UA and DoS Attack

DoS attacks aim to exhaust critical resources such as bandwidth, processing power, or memory, rendering the target inaccessible to legitimate users. DoS attacks are classified based on their execution methods, which typically fall into two categories: resource exhaustion and exploitation of software or protocol weaknesses [13].

IIoT devices operate with a constant exchange of data. In this context, DoS attacks can potentially disrupt normal operations in distributed systems. Aiming at secure information exchange, the OPC UA standard facilitates communication by connecting devices using a Client-Server architecture. In this case, the server identifies the client initiating the connection and allocates resources to maintain it. This process, however, opens opportunities for directed DoS attacks, potentially impacting system availability [14].

To minimize the effects of DoS attacks on OPC UA servers, the OPC Foundation defines authentication and authorization features that can restrict access to computing resources [9]. In addition, features accessible without authentication criteria could be used to increase resilience to network attacks, e.g., `GetEndpoints` and `OpenSecureChannel` are features that are requested before the client authenticates to the server.

III. MATERIALS AND METHODS

To evaluate OPC UA security using the `hping3` and OPC UA Exploitation Framework tools, we implemented a client-server

architecture. Two computers on the network were configured to run the client and server sides, as illustrated in Figure 3, while a third computer, equipped with an Intel Core i7-1255U and 16 GB of RAM, was used to execute the attack using hping3.

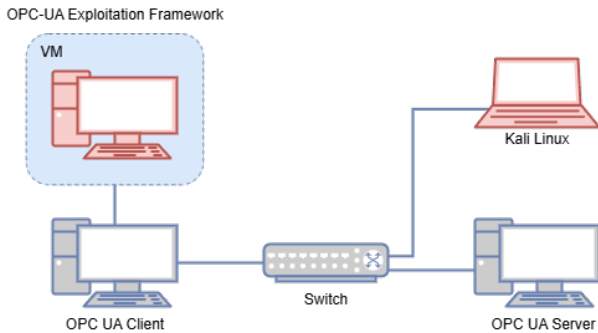


Fig. 3. Network architecture used for the tests.

In this study, Claroty's OPC UA Exploitation Framework is used as a malicious client. The framework was installed on a virtual machine (VM) configured on the OPC UA client machine, equipped with 4 cores and 4 GB of RAM. The OPC UA server used for testing was the Prosys OPC UA Simulation Server version 5.4.6-148, hosted on a computer with an Intel i9-14900F processor and 64 GB of RAM. The client application used for investigate communication was UaExpert Version 1.7.1 540, running on a machine with an Intel i7-7700 processor and 8 GB of RAM.

To conduct the tests, the tools hping3 and Claroty's OPC UA Exploitation Framework were used to launch attacks against the OPC UA server under different SecurityMode configuration settings. The hping3 tool is widely utilized by security professionals for performing DoS attacks and is one of the tools natively available in Kali Linux. Its capabilities allow for executing DoS attacks by sending a customized volume of TCP packets, adjusted according to the intensity of the attack targeted at the device [4]. In our test, we employed the command: `sudo hping3 -S --flood -v -p 4840 -d 120 192.168.1.100`. This command floods TCP packets to port 4840 of the target device.

The OPC UA Exploitation Framework, available on Claroty's ¹, is an advanced OPC UA framework designed for vulnerability research and exploitation, developed by the Claroty Team82. The tool operates as an OPC UA client, built using Python's implementation library to support most structural features. This implementation enables the execution of OPC UA-specific attacks, which can cause denial of service, leak sensitive information, or even execute remote code. It supports up to 13 different server types, including Prosys. We used this tool to launch attacks by arbitrarily invoking the ConditionRefresh method on the OPC UA server. Subsequently, we analyzed the server's varying behavior under different SecurityMode configuration settings.

¹<https://github.com/claroty/opcu-exploit-framework>

IV. RESULTS

The first test was conducted using the widely recognized DoS attack tool hping3, which is natively available in the Kali Linux distribution. A TCP-SYN attack was executed on port 4840, the default port for OPC UA, and port 53530, the custom default port for the Prosys Server, which can be modified in the application settings if necessary. The tests were performed under two configurations: one with the OPC UA server operating without security settings and the other with security enabled, incorporating authentication methods and message signing and encryption. In both scenarios, the server was subjected to high network traffic, causing frequent connection drops between the OPC UA client and server, as shown in Figure 4. However, the test results indicated no significant impact on CPU or RAM usage in either case.

```
Message
Connection status of server 'Host - Servidor' changed to 'ConnectionWarningWatchdogTimeout'.
Connection status of server 'Host - Servidor' changed to 'Connected'.
[uastack] OpcUa_Channel_ResponseAvailable: Request failed! (0x800A0000)
Connection status of server 'Host - Servidor' changed to 'ConnectionWarningWatchdogTimeout'.
[uastack] OpcUa_Channel_ResponseAvailable: Request failed! (0x800A0000)
Connection status of server 'Host - Servidor' changed to 'ConnectionErrorApiReconnect'.
[uastack] OpcUa_Channel_ResponseAvailable: Request failed! (0x80AD0000)
[uastack] OpcUa_Channel_ResponseAvailable: Request failed! (0x80AD0000)
[uastack] OpcUa_Channel_ResponseAvailable: Request failed! (0x80AD0000)
Error: Publish failed [status=0x80ad0000]
Error: Publish failed [status=0x80ad0000]
Error: Publish failed [status=0x80ad0000]
```

Fig. 4. Server connection drops during the attack.

The subsequent tests were conducted using the OPC UA Exploitation Framework tool. This tool enables the "Unlimited Condition Refresh" attack, which exploits a DoS vulnerability. The tests involved varying the OPC server's Security Mode, Security Policy, and Authentication Token. Alongside the constant loss of connection to the server, the attack also impacted CPU and RAM usage, as depicted in Figure 5. Notably, in the worst-case scenario with Security Mode set to None, memory usage increased by nearly 200% compared to the state before the attack. On the other hand, with SecurityMode Sign or SignEncrypt enabled, the OPC UA Exploitation Framework tool was unable to connect to the server, preventing the attack entirely.

Figure 6 illustrates the OPC UA communication packets exchanged between the Exploitation Framework tool and the server. When authentication methods are enabled, the server generates a ServiceFault message with the StatusCode *BadIdentityTokenInvalid*, preventing the connection from being activated. Similarly, when encryption is applied, the server responds with *BadSecurityModeRejected*, indicating that the SecurityMode does not meet the server's requirements.

The attack achieves partial success when invalid authentication credentials are used. In this case, the server returns a ServiceFault with *BadIdentityTokenInvalid*, which blocks session activation but permits limited interaction with the server. Conversely, encryption configured with an unsupported SecurityMode triggers a *BadSecurityModeRejected* response, terminating the connection entirely.

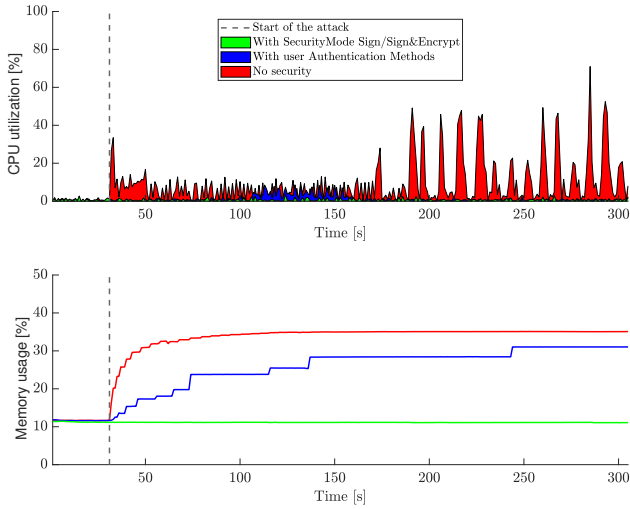


Fig. 5. CPU Load and RAM usage during attack.

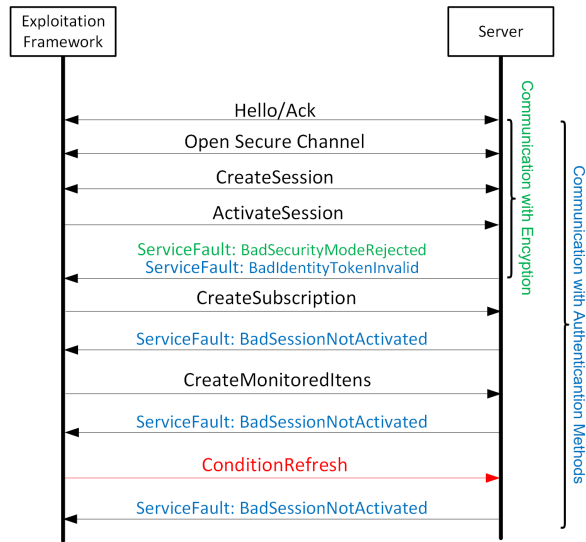


Fig. 6. OPC-UA message exchange between the Exploitation Framework tool and the server.

V. CONCLUSION

In this paper, we evaluated the behavior of an OPC UA server under DoS attacks across three security configurations. While OPC UA offers robust security mechanisms, its flexibility in configuration allows for tailored setups, which may inadvertently result in insufficient security levels. Using both hping3 and the OPC UA Exploitation Framework, we demonstrated that while both tools can disrupt availability, the OPC UA Exploitation Framework facilitates more advanced and impactful attacks. Our findings highlight that properly configured security mechanisms, such as Sign and SignEncrypt, effectively mitigate advanced DoS attacks, emphasizing

the critical importance of implementing appropriate security settings in OPC UA environments.

Future work will investigate the security configurations of the Global Discovery Server (GDS) and the PubSub architecture, focusing on their resilience to attacks executed with security tools.

ACKNOWLEDGMENT

This work has been funded by Petrobras through Cooperation Agreement No. 0050.0126772.24.9 and by the project Investigação e experimentação tecnológica para infraestrutura de aplicações em plataformas de sensoriamento inteligente supported by CENTRO DE COMPETÊNCIA EMBRAPPI VIRTUS EM HARDWARE INTELIGENTE PARA INDÚSTRIA, with financial resources from the PPI HardwareBR of the MCTI grant number 055/2023, signed with EMBRAPPI. Thiago Euzébio acknowledges the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) through grant 306394/2021-9.

REFERENCES

- [1] Jonathan Tobias Da Silva, Andre Luis Dias, and Ivan Nunes Da Silva. A survey on OPC UA protocol: overview, challenges and opportunities. In *2023 15th IEEE International Conference on Industry Applications (INDUSCON)*, pages 1523–1530. IEEE, 2023.
- [2] Ercan Nurcan Yılmaz, Bünyamin Ciyilan, Serkan Gönen, Erhan Sindiren, and Gökçe Karacayılmaz. Cyber security in industrial control systems: Analysis of dos attacks against PLCs and the insider effect. In *2018 6th international istanbul smart grids and cities congress and fair (icsg)*, pages 81–85. IEEE, 2018.
- [3] Julien Polge, Jeremy Robert, and Yves Le Traon. Assessing the impact of attacks on opc-ua applications in the industry 4.0 era. In *2019 16th IEEE annual consumer communications & networking conference (CCNC)*, pages 1–6. IEEE, 2019.
- [4] Vaishnavi Varadarajan. Security analysis of OPC UA in automation systems for iiot, 2022.
- [5] Dong-Hyuk Shin, Ga-Yeong Kim, and Jeck-Chae Euom. Vulnerabilities of the open platform communication unified architecture protocol in industrial internet of things operation. *Sensors*, 22(17):6575, 2022.
- [6] Clarity. OPC UA deep dive: A complete guide to the OPC UA attack surface, April 2023. Accessed: 2025-01-09.
- [7] Marc Ladegourdie and Jonathan Kua. Performance analysis of OPC UA for industrial interoperability towards industry 4.0. *IoT*, 3(4):507–525, 2022.
- [8] R Donald Bartusiak, Stephen Bitar, David L DeBari, Bradley G Houk, Dennis Stevens, Bridget Fitzpatrick, and Patrick Sloan. Open process automation: A standards-based, open, secure, interoperable process control architecture. *Control Engineering Practice*, 121:105034, 2022.
- [9] Wolfgang Mahnke, Stefan-Helmut Leitner, and Matthias Damm. *OPC unified architecture*. Springer Science & Business Media, 2009.
- [10] OPC Foundation, “OPC unified architecture specification”, part 4. Available at: <https://reference.opcfoundation.org/Core/Part4/v104/docs/7.36>. Accessed: 09-01-2025.
- [11] Markus Dahlmanns, Roman Matzutt, Chris Dax, and Klaus Wehrle. Collectively enhancing iot security: A privacy-aware crowd-sourcing approach. In *International Symposium on Foundations and Practice of Security*, pages 15–27. Springer, 2023.
- [12] Johannes vom Dorp, Sven Merschjohann, David Meier, Florian Patzer, Markus Karch, and Christian Haas. OPC UA security analysis. In *Bundesamt für Sicherheit in der Informationstechnik (BSI)*, pages 1–69. Federal Office for Information Security, 2022.
- [13] Vinícius De Miranda Rios, Pedro RM Inácio, Damien Magoni, and Mário M Freire. Detection and mitigation of low-rate denial-of-service attacks: A survey. *IEEE Access*, 10:76648–76668, 2022.
- [14] OPC Foundation, “OPC unified architecture specification”, part 2. Available at: <https://reference.opcfoundation.org/Core/Part2/v104/docs/5.1.2.3>. Accessed: 09-01-2025.